Challenges and Lessons from Human-Led Formalization

Weil Conjectures Project

October 2026

The Weil conjectures project relies heavily on category theory, particularly derived categories. This has presented challenges at two different levels: technical difficulties, most notably with the performance of Lean's kernel, and broader questions about how to choose and organize the mathematical constructions underlying the formalization. Here we describe several concrete problems we encountered, how we addressed them, and what they taught us about formalization on this scale.

1 Kernel Checking and Performance

Our extensive use of category theory has exposed several serious performance bottlenecks, the most severe of which arose during kernel checking rather than tactic elaboration. In such cases, Lean could elaborate a proof term relatively quickly, yet the kernel would take an unexpectedly long time to check it or establish a required definitional equality. Although we do not have a complete explanation for every slowdown, the following cases illustrate ways to avoid some of the most costly comparisons.

1.1 Timeout in the Base-Change Morphism

Our first major example arose while proving an `_app` lemma for the base-change morphism, viewed as a natural transformation. For a commutative square, we construct this morphism in three steps using the adjunction $f^* \dashv f_*$:

$$\begin{array}{ccc} X' & \xrightarrow{g'} & X \\ {\scriptstyle f'}\downarrow & & \downarrow{\scriptstyle f} \\ Y' & \xrightarrow[g]{} & Y \end{array}$$

$$\mathrm{Ex} : g^* R f_* \to R f'_* f'^* (g^* R f_*) \to R f'_* g'^* (f^* R f_*) \to R f'_* g'^*.$$

The most complicated part was the middle step, which transports the composition of functors across the commutative square and set up `_app` lemma.

```
def Etale.baseChangeSquareStep (sq : CommSq g' f' f g) :
    (R(f)_*[A]+ ⋙ (g)^*[A].mapDerivedCategoryPlus) ⋙
        ((f')^*[A].mapDerivedCategoryPlus ⋙ R(f')_*[A]+) ≅
    (R(f)_*[A]+ ⋙ (f)^*[A].mapDerivedCategoryPlus) ⋙
        ((g')^*[A].mapDerivedCategoryPlus ⋙ R(f')_*[A]+) :=
  Functor.associator _ _ _ ≪≫ Functor.isoWhiskerLeft R(f)_*[A]+
    ((Functor.associator _ _ _).symm ≪≫ Functor.isoWhiskerRight
      (pullbackSquareComparison A sq) R(f')_*[A]+ ≪≫
        Functor.associator _ _ _) ≪≫ (Functor.associator _ _ _).symm
```

Code excerpt 1: The original square-transport construction.

```
lemma Etale.baseChangeSquareStep_app (sq : CommSq g' f' f g)
    (K : DerivedCategory.Plus (Sheaf X.smallEtaleTopology A)) :
    ((Etale.baseChangeSquareStep A sq).app K) =
      R(f')_*[A]+.mapIso ((pullbackSquareComparison A sq).app (R(f)_*[A]+.obj K))
```

We expected `simp` to complete this proof. Elaboration was relatively quick, but the kernel timed out. Even after simplifying the expression, proofs using either `simp` or `rfl` still timed out. This was our first indication that the bottleneck lay in kernel checking, not in finding a proof through elaboration.

Our initial workaround was to replace the specific pullback and pushforward functors with abstract functors. We proved a general square-transport construction and its `_app` lemmas at that level, then specialized those results to base change:

```
def squareTransport {G1 : C2 ⥤ C3} {G2 : C3 ⥤ C4} {G3 : C2 ⥤ C3'}
    {G4 : C3' ⥤ C4} (R : C1 ⥤ C2) (R' : C4 ⥤ C5) (θ : G1 ⋙ G2 ≅ G3 ⋙ G4) :
    (R ⋙ G1) ⋙ (G2 ⋙ R') ≅ (R ⋙ G3) ⋙ (G4 ⋙ R')
```

Code excerpt 2: An abstract square-transport construction.

This resolved the immediate problem, but we encountered a similar issue later when proving an `_app` lemma for base change of the exceptional pushforward $f_!$. In that construction, the base-change morphism appeared as one component of a longer composite:

```
def Etale.lowerShriekBaseChange (c : Compactification f)
    (sq : IsPullback g' f' f g) :
    Rf_!(c)[A]+ ⋙ D+((g)^*[A]) ⟶ D+((g')^*[A]) ⋙ Rf_!(c.baseChange sq)[A]+ :=
  (Functor.associator _ _ _).hom ≫ Functor.whiskerLeft D+((c.i)_!open[A])
    (Etale.baseChange A (c.baseChange_isPullback_g sq).toCommSq) ≫
  (Functor.associator _ _ _).inv ≫ Functor.whiskerRight
    (Etale.openImmersionLowerShriekBaseChangeIso A
      (c.baseChange_isPullback_i sq)).hom R+(((c.baseChange sq).g)_*[A]) ≫
  (Functor.associator _ _ _).hom
```

Code excerpt 3: Base change for the exceptional pushforward.

This made us reconsider the case-specific workaround: while it avoided an individual timeout, it did not address the underlying difficulty in a principled or sustainable way. We therefore marked `Etale.baseChange` with `@[no_expose]`. In this case, preventing downstream code from unfolding that definition was enough to avoid the expensive kernel reduction. This did not deprive downstream proofs of the necessary information: the existing `_app` lemmas already characterized the natural transformation at the level where it was used.

This is a reusable technique: marking a construction that already characterized by related lemmas `@[no_expose]`. However, it isn't widely applicable, since there need to be suitable declaration with enough characterization to work on, for example even until now we didn't find a way to replace the workaround by marking some `@[no_expose]`, because it is too close to the basic constructions of pullback and pushforward. Thus, `@[no_expose]` is particularly attractive when the existing API already describes the declaration sufficiently, but its implementation is likely to cause problems if exposed downstream.

1.2 Slow Derived Tensor

A second issue arose while constructing the instance stating that the derived tensor product of *étale* sheaves is a left derivation of the ordinary tensor product. More precisely, we were constructing an `IsLeftDerivedFunctor`$_2$ instance for `curriedTensor (DerivedCategory (Sheaf`

`X.smallEtaleTopology A))`. Even providing a proof with exactly the expected type led to very slow kernel checking. We found that changing `Functor.IsLeftDerivedFunctor`$_2$ from an `abbrev` to an `@[reducible] def` resolved the issue.

This is because instead of `abbrev` that would expand in kernel directly, because any `def` is marked as regular, the kernel could use same-head argument comparison through lazy delta reduction i.e. doing `congr`, checking whether they are directly syntactically equal. (The `@[reducible]` attribute was only retained to preserve the intended elaboration behavior.)

We are especially grateful to Professor Joël Riou for providing the first construction of the derived tensor product (in Mathlib4 PR 25848). Based on this, here we managed to improve the performance of the surrounding development. As reflected in Mathlib4 `IsLeftDerivedFunctor`$_2$, the multivariate derived-functor definitions are now `@[reducible] def`s rather than `abbev`.

1.3 Timeout in the Derived-Completion Cone

Another kernel timeout arose in an earlier development of derived completion, which has since been superseded. Although that implementation was not retained, diagnosing the timeout revealed a different source of expensive definitional equality. Simply declaring the derived-completion cone was enough to trigger the problem, even with its value replaced by `sorry`:

```
noncomputable def DerivedCategory.derivedCompletionCone
    (K : DerivedCategory (Sheaf J (ModuleCat.{u + 1} R))) :
    Cone (DerivedCategory.quotPowFunctor J I ⋙ tensorLeft K) :=
  sorry
```

Code excerpt 4: A declaration that already caused a timeout.

In our investigation, `Kernel.check(type)` and `Kernel.check(value)` both completed in a reasonable time, whereas `Kernel.isDefEq(inferredType, declaredType)` timed out. The comparison involved proof terms that were not syntactically identical after Lean's `abstractNestedProofs` processing. Such differences are ordinarily harmless because of proof irrelevance, but the kernel may still have to compare the propositions in which the proofs live. Here those propositions involved large proof-carrying instance expressions, including the `MonoidalCategory` instance on `DerivedCategory`, so the comparison could descend into a substantial amount of structure.

One useful diagnostic experiment was to pass the derived-category `MonoidalCategory` instance as an explicit parameter, rather than having Lean infer the concrete instance; the corresponding check then completed quickly. The final solution is direct and preserved the intended construction: we gave the indexing diagram `DerivedCategory.quotPowFunctor J I ⋙ tensorLeft K` its own named definition and used that name in the type of the cone. Even an `abbrev` helped in this case. The named diagram provided a stable point of reference, so the kernel no longer had to compare independently expanded versions of the entire construction, including the proofs nested within it.

1.4 Slow Monoidal Pullback

While working on the monoidality of pullback functors, we encountered two further, complementary patterns of expensive kernel conversion. Addressing both was necessary to resolve the performance problems in the file.

First, comparing expressions beneath structure projections can be very expensive even when the underlying structures are cheaply definitionally equal. In our case, the kernel struggled with components of two natural transformations of the form `(`τ`.app X).app Y`, although the natural transformations themselves could be compared efficiently. We avoided asking the kernel to rediscover that equality after successive applications of `NatTrans.app`. Instead, we introduced a general

congruence-style lemma that transports a componentwise equation along a propositional equality of natural transformations:

```
lemma app_app_comp_of_eq (h : (τ.app X).app Y ≫ f = k) (σ : F → G)
    (hσ : σ = τ) : (σ.app X).app Y ≫ f = k :=
  hσ ▶ h
```

The original proof ended with `exact h`, which required the kernel to establish a potentially expensive definitional equality beneath two applications of `NatTrans.app`. With the new lemma, the final `rfl` compares the natural transformations themselves, before taking their components. The essential optimization was therefore to move the equality obligation above the expensive projections, rather than merely splitting the proof into smaller lemmas.

Second, passing a compatibility `Prop` to a generic constructor could trigger an expensive type conversion. Although the theorem's type and the constructor's expected binder proposition were definitionally equal, their elaborated expressions are not directly syntactically equal. Unfolding and comparing the large propositions could lead to extensive recursive reduction. We addressed this by giving the compatibility condition a stable name using an ordinary `def`.

```
def LaxSquareHδ (δG : ...) (e : ...) : Prop :=
  ∀ a b, (δG.app ...).app ... ≫ ... = ...

noncomputable def ofLaxSquareHδ
    ... (hδ : LaxSquareHδ δG e) ... :=
  Functor.OplaxMonoidal.ofLaxSquare ... hδ ...
```

We also supplied a bridge theorem whose declared type used precisely `LaxSquareHδ`. This served two related purposes. First, when the expected and supplied types shared the same regular-definition head, the kernel could try comparing their arguments before unfolding the potentially large proposition. Second, the bridge theorem isolated the remaining conversion in a separate kernel-checking context, where it proved substantially cheaper. An `abbrev` did not give the same benefit, because it led to more aggressive unfolding.

These remedies addressed different bottlenecks. The first changed where a proof's equality was established, while the second made the type of a proof argument easier for the kernel to match against the constructor's expected binder type. Together they made the entire file as much as five times faster to check; applying either strategy alone still left a significant type-checking bottleneck.

Taken together, these cases gave us a more systematic way to investigate kernel slowdowns and made it possible for coding agents to contribute to performance improvements. The broader lesson is that the mathematical content of a construction need not change in order to make it tractable for the kernel: a well-chosen abstraction or interface can prevent an otherwise expensive comparison. We still do not fully understand every slowdown, but these experiences have made such problems far more manageable.

2 Mathematical Design Choices in Formalization

Beside pure technical issues, we learned that decisions related to mathematical design are crucial to formalizing correctly, choosing a workable setup, and making progress efficiently. These decisions are often implicit in conventional mathematical writing, but a formalization project must make them explicit at the right time.

There are hidden or vague parts in natural language that needs to be pinned down, for example universe issue, which wouldn't be exhibited in any natural language proof. Initially, we considered `X : Scheme.{u}` and a Grothendieck abelian coefficient category `A` of universe level `u`. In this

setting, we could construct the sheafification of `A`-valued presheaves on the *étale* site of `X`. However, the corresponding construction for the *pro-étale* site could not use the same universe assignment. Unlike the *étale* site, the *pro-étale* site over `X` is not essentially small, and we did not have an analogous small dense subsite, such as `AffineEtale`, for case-specific hack. We therefore needed the target Grothendieck abelian category of universe level `u + 1`. Because this difference had not been anticipated, accommodating it required changes to signatures and constructions throughout a substantial part of the development. Identifying such constraints immediately after the blueprint stage, before large-scale formalization began, would have saved considerable rework.

Apart from corners that haven't being considered in natural language, the blueprint itself should also be adjusted for formalization, it cannot always use the same shorthand as a blueprint for human readers. For example, despite its suggestive notation, $Rf_!$ is not simply the total right-derived functor of the underived $f_!$. While I was not sufficiently familiar with this part of the mathematics thus not aware of this difference, Opus also didn't notice and correct the implementation either. Similar ambiguities arise in the six-functor notation: identical symbols can refer to functors constructed with different coefficient categories, sometimes by quite different methods. A paper can often leave the coefficient category implicit, but doing so in a formalization blueprint can send an agent down the wrong path. Specifying the intended construction and disambiguating overloaded notation are therefore substantive parts of preparing a useful blueprint, not merely matters of presentation.

The geometric assumptions should also be decided early. Restricting the six operations to bounded constructible derived categories requires carefully chosen geometric and coefficient hypotheses, involving conditions such as quasi-excellence and an ℓ-coprime base scheme. For the part of the theory relevant to our project, choosing to work with schemes that are separated and of finite type over a field makes many of these hypotheses directly manageable. Without explicitly deciding on this particular separated, finite-type over field setup, an agent might select assumptions that are too strong, too weak, or incompatible with results needed later. Making that concrete choice early was therefore important.

Other than the final setup we usually use, we also need to take care of the way how we construct them. For example, the six-functor formalism with $\overline{\mathbb{Q}}_\ell$ coefficients cannot simply be obtained by restricting the formalism for sheaves valued in `ModuleCat`, without considering the topology on the coefficient ring. Our initial plan was to construct the category of $\overline{\mathbb{Q}}_\ell$-sheaves as a 2-colimit of categories of sheaves with coefficients in finite extensions E, while retaining a forgetful functor that would realize the resulting objects as genuine $\overline{\mathbb{Q}}_\ell$-sheaves. Besides the difficulty of formulating and implementing the required 2-colimit, this approach encountered a more structural problem: because the category of $\overline{\mathbb{Q}}_\ell$-sheaves has too many isomorphisms, it is not simply the union of the essential images of the inclusion functors, the morphisms between objects would therefore be more complicated. This motivated our move to a framework based on ring objects and module objects.

Once we adopted the module-object perspective, we sought a more uniform way to handle coefficients in $\mathcal{O}_E$, E, and $\overline{\mathbb{Q}}_\ell$. To construct the six functors uniformly, we needed a compatible Cartesian system of ring objects. A mathematical expert on the project pointed out the appropriate organizing idea: these ring objects should arise by pullback from a point, with the universal point corresponding to condensed rings. With this setup, we could place the unbounded six-functor constructions for all these coefficient systems in a common framework.

Finally, there are some lessons on formalizing efficiently, especially for choosing the right level of generality. Early on, we wanted a uniform setting for the coefficient categories that might arise in étale sheaf theory, such as `ModuleCat` and `Ab`. Although mathematically natural, that generality imposed a significant upfront cost. In retrospect, a better sequence would have been to implement the concrete constructions for `ModuleCat` and `Ab` first, and then return to the question of how to treat them uniformly within a suitably general setup.

Some abstractions turned out to be unhelpful altogether. We initially tried to organize the six-functor formalisms for the *étale* and *pro-étale* sites within a single framework. Their Lean implementations diverged at a very early stage, because the *pro-étale* setting uses quasi-compact covers, rather than relying only on joint surjectivity. While the general site-level pullback and pushforward constructions already occupied the natural common layer, there are no good abstractions to do, we therefore refactored the development into two frameworks directly specifying to the respective sites. This happened because at the early stage, we focused too much on creating big blocks for agents to work on, in this case they are *étale* proper base change and smooth base change. Even a modest amount of work on the *pro-étale* side at that stage would probably have exposed the mismatch much earlier. The lesson was not to avoid abstraction, but to test a proposed common framework against the distinct cases in the first place before investing heavily in it.